

轮 趣 科 技

WHEELTEC 自平衡自行车 电路与程序设计手册

推荐关注我们的公众号获取更新资料



版本说明:

版本	日期	内容说明
V1.0	2022/06/23	第一次发布

网址: www.wheeltec.net

序言

本文档主要讲解 WHEELTEC Z100 舵机自平衡自行车的电路和软件开发设计思路与实现，电路设计包括各电子模块的说明和原理，软件开发包括 STM32 单片机资源设计分布和各部分程序的实现原理。

最后提供了自行车平衡控制的流程图，关于 LQR 平衡控制原理部分的内容在另一个文档【3.基于车把转向控制的自平衡无人自行车.docx】内。

目录

序言	2
1. 电路设计	4
1.1 单片机最小系统设计	4
1.2 陀螺仪电路设计	4
1.3 电机驱动电路设计	5
1.4 电机编码器	6
1.5 电位器电路设计	7
1.6 舵机电路设计	8
1.7 电源电路设计	8
2. 软件开发	9
2.1 小车主程序设计	9
2.2 小车控制中断任务程序设计	10
2.3 电机与舵机驱动程序设计	10
2.4 电机编码器读取程序设计	11
2.5 蓝牙程序设计	12
2.6 ADC 采集电池电压与电位器程序设计	13
3. 小车控制流程图	14

1. 电路设计

1.1 单片机最小系统设计

舵机小车采用 STM32F103C8T6 作为主控，其原理图如图 1-1 所示。主控的输入电压为 5V 或 3.3V，电源部分采用 AMS1117 芯片，将 5V 转换为 3.3V 提供给 STM32 芯片和外部模块使用。此外，主控板设有一键下载的串口电路、一个用户按键和一个复位按键、一个可编程 LED 和一个电源指示灯。

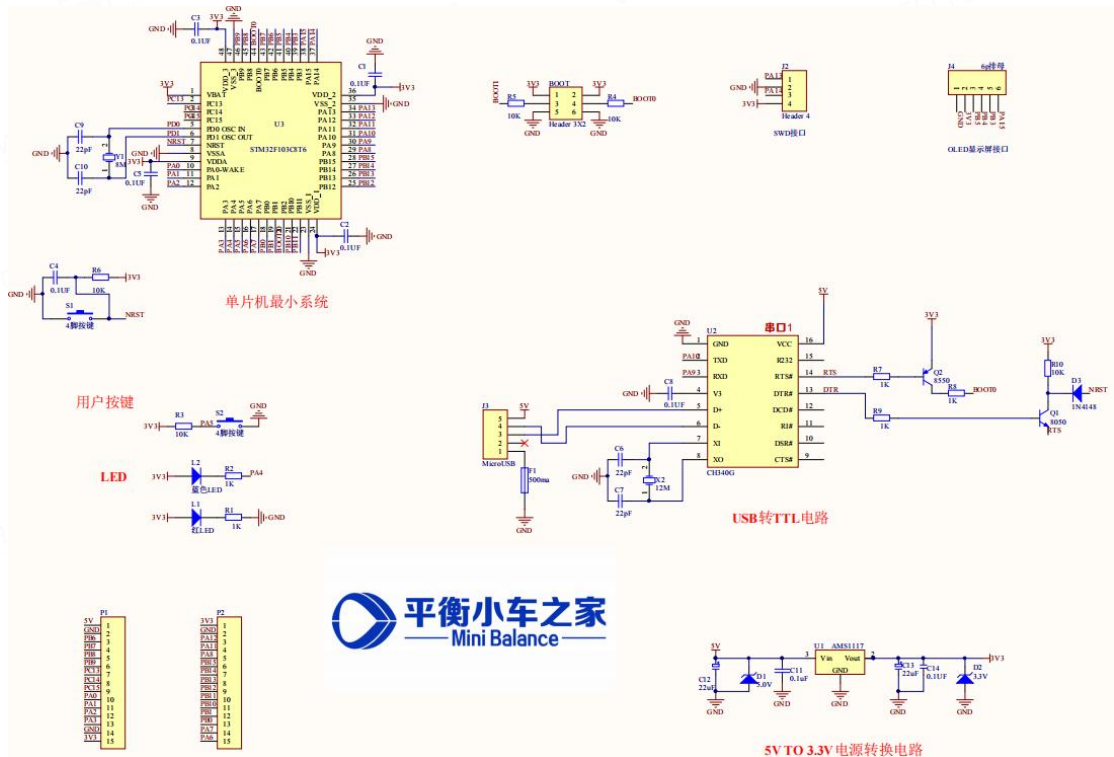


图 1-1 STM32 最小系统设计

1.2 陀螺仪电路设计

小车使用 MPU6050 芯片作为陀螺仪传感器，该芯片内带 3 轴角速度和 3 轴加速度传感器，使用 IIC 协议通信，其内部自带数字运动处理器（DMP）硬件加速引擎，可以直接输出 6 轴的姿态解算数据；为了满足需要和测量的准确，传感器提供了不同的量程，陀螺仪范围可从 $\pm 250^\circ$ 到 $\pm 2000^\circ$ ，加速度计范围可从 $\pm 2g$ 到 $\pm 16g$ ，满足大部分使用场景。其原理图如图 1-2 所示。其中 AD0 引脚可以改变该芯片在 IIC 通信的地址，AD0 接入 GND 时地址为 0x68，接入 VCC 时为 0x69；INT 引脚是芯片的中断引脚，当芯片数据就绪后，该引脚会由高电平变为低电平；

数据就绪频率与芯片配置的采样频率相同；SDA、SCL 是 IIC 通信引脚，在这里接入了单片机的普通 IO，所以程序编写时将会通过模拟 IIC 的方式与该芯片进行通信。

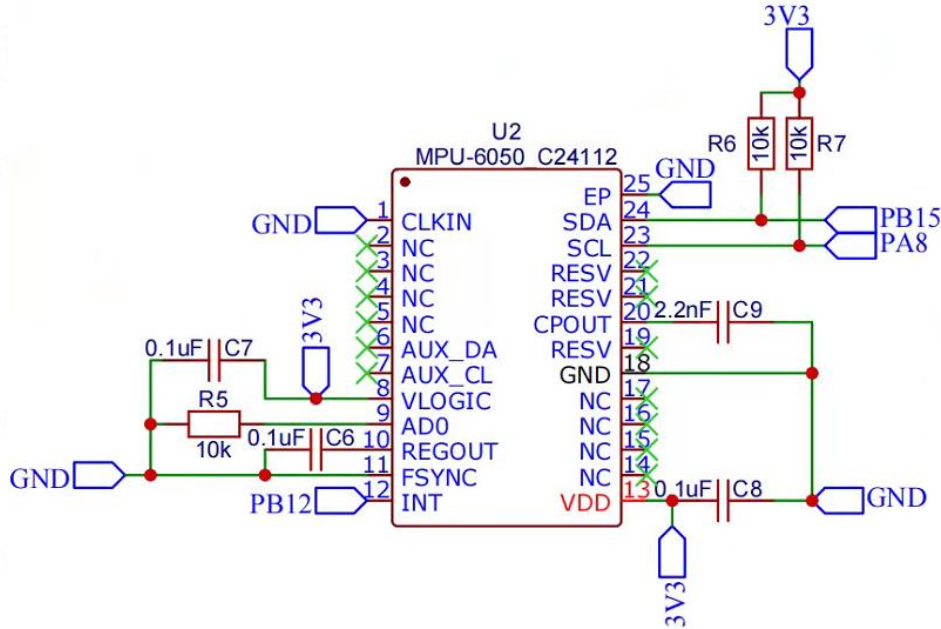


图 1-2 MPU6050 连接示意图

1.3 电机驱动电路设计

标准版舵机小车只使用到一个电机，用于小车后轮驱动。驱动芯片采用 AT8236，其内部是一个由 4 个 mos 管组成的 H 桥结构，示意图如图 1-3 所示。

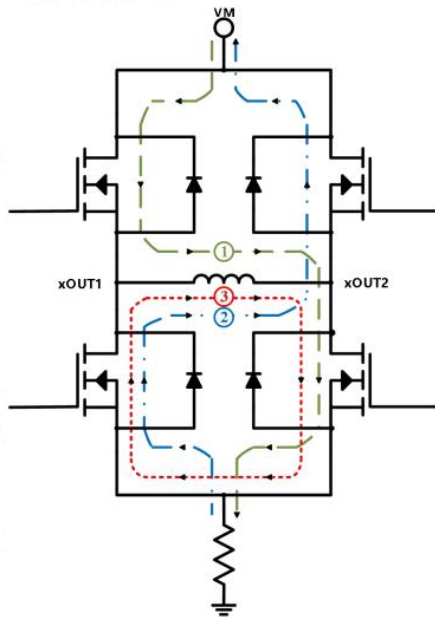


图 1-3AT8236 内部 H 桥示意图

该芯片可以使用普通 TTL 电平驱动电机（无法对电机进行调速），也可使用 PWM 输入来驱动电机，在这里主要用到 PWM 驱动，原理图如图 1-4 所示。AT8236 的 PWM 控制真值表如下：

表 1-1 AT8236 真值表

IN1	IN2	功能
PWM	0	正传 PWM，快衰减
1	PWM	正传 PWM，慢衰减
0	PWM	反传 PWM，快衰减
PWM	1	反传 PWM，慢衰减

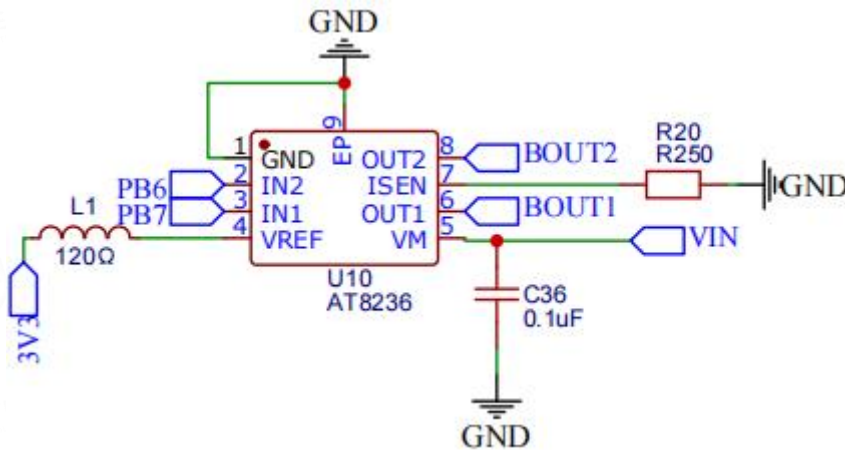


图 1-4 AT8236 原理图

1.4 电机编码器

小车上使用的电机集成了 GMR 编码器，电机的转动可使编码器向外输出脉冲，脉冲分为 AB 两相，且两个脉冲之间带有 90° 相位差。示意图如图 1-5 所示。

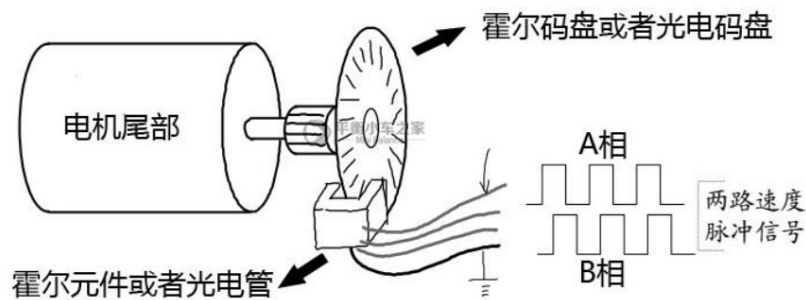


图 1-5 电机旋转使得编码器向外输出脉冲

我们可以通过读取脉冲计数，从而获得电机的转速，对电机进行闭环控制。其中脉冲数与电机减速比有如下的关系：

电机旋转一圈编码器输出的脉冲个数 = 电机减速比 × 编码器精度(线数)

在程序中我们使用 STM32 的定时器来读取编码器输出的脉冲数，其优势是不需要占用 CPU 的资源，由硬件读取，且能够配置为 4 倍频，提高了编码器的精度。所以我们需要对外引出定时器的两个引脚，并且为编码器提供电源，其原理图如图 1-6 所示。

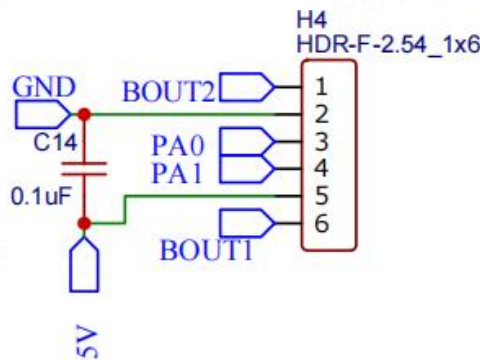


图 1-6 电机编码器与正负极接口

1.5 电位器电路设计

舵机小车包含两个电位器，一个用于调节小车的舵机初始位置，一个用于调节小车的平衡中值。调节这两个电位器的目的均是为了小车能够平衡，而不需要重新修改程序，方便用户使用。其原理图如图1-7所示。图中使用的电位器最大阻值为10k，同时串联了一个10k 电阻，因此电位器上的电压范围是0~1.65V。

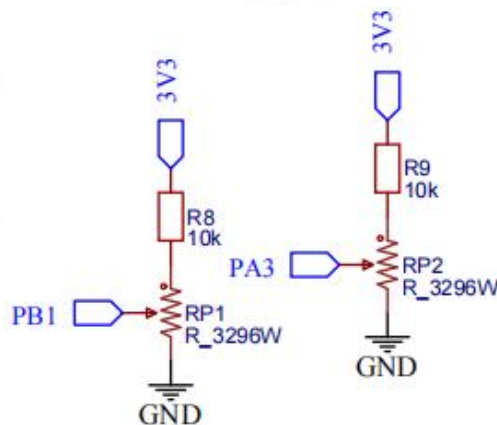


图1-7 电位器原理图

1.6 舵机电路设计

舵机驱动只需要一路 PWM 即可完成控制，其中 PWM 占空比与舵机旋转角度关系如下图1-8所示。需要注意的是，驱动舵机的 PWM 频率一般为50Hz。

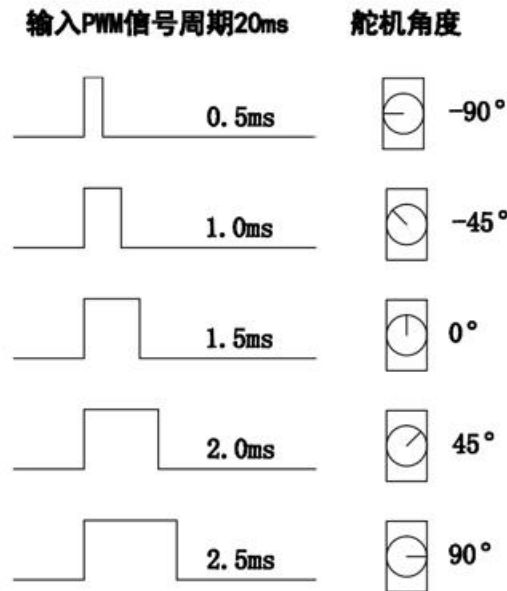


图1-8 输入舵机的 PWM 占空比与角度关系图

因此舵机的控制只需要引出电源线和一個定时器引脚即可。其原理图如图1-9所示。

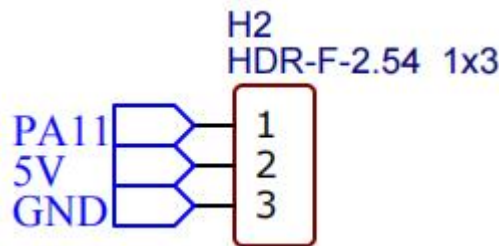


图1-9 舵机接口原理图

1.7 电源电路设计

小车电源部分采用 RT8289GSP 降压芯片进行稳压，该芯片支持5~32V 的宽电压输入，经过芯片稳压后可对外输出5V5A 的功率；此外，在电源输入端串联了一个10k 和一个1k 的电阻，对电池电压进行分压，方便单片机进行采样计算，其原理图如图1-10所示。需要注意的是，电源电压会输入到电机驱动芯片进而施

加在电机上，施加给小车电机的电压不宜过高，建议电压范围在7~12V。小车上采用2s 航模电池供电，满电状态为8.4V。

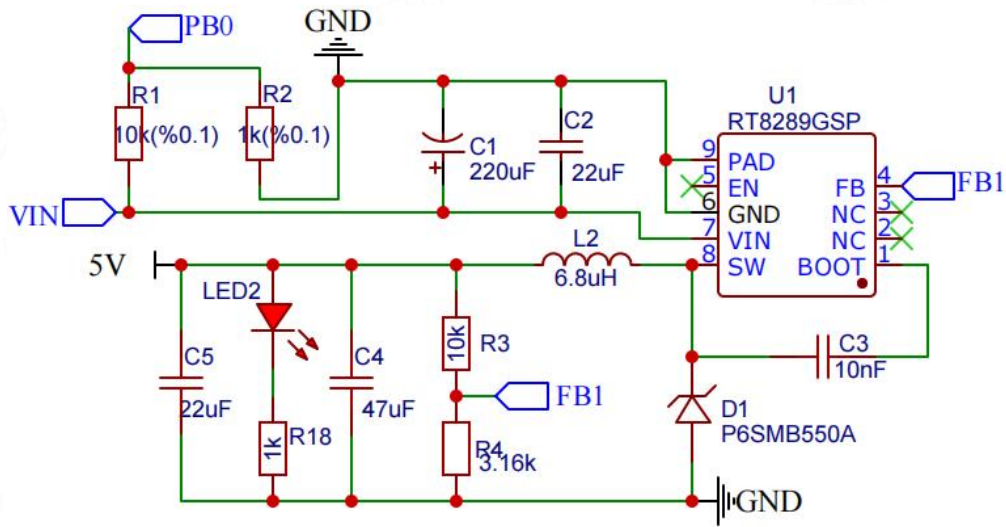


图1-10 电源电路原理图

2. 软件开发

2.1 小车主程序设计

小车所有外设的初始化均在主程序中执行，下面对使用到的单片机资源进行说明：

USART1：用于发送数据到上位机，显示波形

USART3：用于蓝牙 app 与小车之间相互通信

TIM4：使用 TIM4CH1 和 TIM4CH2 产生 PWM 驱动后轮电机

TIM2：初始化为正交编码器模式，用于采集电机编码器输出的脉冲数

ADC：使用 ADC1 的通道 3、8、9，分别用于采集修改机械中值的电位器、电池电压、修改舵机初始位置的电位器

TIM1：使用 TIM1CH4 产生 PWM 驱动舵机

EXTI：配置 PB12 映射到外部中断线 12，由 MPU6050 的 INT 引脚触发 5ms 外部中断

IIC：小车没有使用到硬件 IIC，而是通过 PA8、PB15 来模拟 IIC 通信

SPI：小车没有使用到硬件 SPI，通过 PB3、4、5 和 PA15 模拟，用于 OLED 显示

SWD：关闭了 JTAG 功能，开启了 SWD 调试功能，可在线调试、下载程序

初始化完上述的单片机资源后，开始进入死循环函数。在死循环里通过标志位来决定小车的显示模式，分为 app 数据发送+oled 刷新和上位机显示波形两种模式可选择，模式的切换由按键改变。此外，当蓝牙发送掉电保存指令时，死循环里的数据写入标志位会被置 1，这时会把小车的 pid 参数写入 Flash，实现掉电保存功能。

2.2 小车控制中断任务程序设计

小车的控制中断任务由 MPU6050 触发的外部中断来提供，其原理是：在配置好 STM32 外部中断的前提下，设定 MPU6050 的采集频率后，并开启 MPU6050 中断功能，此时 MPU6050 的 INT 引脚将会根据采集频率来触发 STM32 的外部中断。

外部中断的配置在 main 函数的 MiniBalance_EXTI_Init()完成，中断的频率可通过下面的宏定义修改。位置位于 MPU6050.c 文件。

```
#define DEFAULT_MPU_HZ (200)
```

在 5ms 的外部中断任务中，单片机会采集电池电压、采集两个电位器的数据、获取小车姿态信息、读取编码器数据等操作，并通过这些数据进行 PID+LQR 计算，最终把计算结果输出到电机、舵机中。

2.3 电机与舵机驱动程序设计

在 1.3 小节中已知电机真值表和电机驱动原理图，我们需要把 TIM4CH1、TIM4CH2 配置成 PWM 输出模式来控制电机的正反转与转速。其中初始化过程在 main 函数的 MiniBalance_PWM_Init(7199,0)函数里，具体配置过程这里不展开，配置结果是将 TIM4 的通道 1、2 配置成了 PWM1 模式，PWM 的频率为 10KHz，其中 0~7200 对应 PWM 的占空比 0~100%。

在 1.6 小节已知舵机的控制方式以及舵机的控制接口为 PA11，所以需要把 TIM1CH4 配置成 PWM 输出模式，由 main 函数中的 Servo_PWM_Init(9999,144)函数配置，配置结果是 PA11 输出频率约为 50Hz 的 PWM 信号，0~10000 对应 0~100%的占空比。

定时器初始化完成后，下面根据电机驱动的真值表开始编写电机的正反转控制以及调速程序和舵机的控制程序。下面是实现过程：

其中 PWMA1 对应 PB7、PWMA2 对应 PB6。SERVO 对应 PA11，这 3 个宏定义修改的均是各自对应的 PWM 占空比，实现电机速度控制、舵机角度控制。

```
void Set_Pwm(int moto1)
{
    if(moto1>0)        PWMA1=7200,        PWMA2=7200-moto1;
    else                PWMA2=7200,        PWMA1=7200+moto1;

    SERVO=Servo + SERVO_BIAS;    //舵机控制
}
```

2.4 电机编码器读取程序设计

在 1.4 小节已知编码器的特性，我们需要一个定时器来计算编码器 AB 两项的脉冲个数，由编码器接口原理图可知，我们需要配置 TIM2 为正交编码器模式，需要注意的是，定时器只有通道 1 和通道 2 有编码器计数功能。配置过程在 main 函数的 Encoder_Init_TIM2()实现，配置结果为将 TIM2 定时器配置成了编码器模式 3，即在每一个脉冲边缘编码器都能计数，从而实现 4 倍频，提高了编码器的精度。下图 2-1 演示了单片机的定时器是读取编码器脉冲的过程。

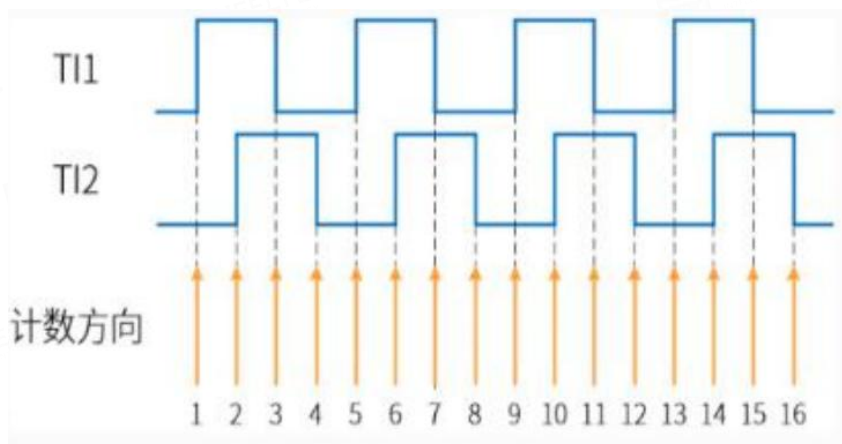


图2-1 定时器读取编码器脉冲示意图

注：当电机的旋转方向发生改变后，TI1 和 TI2 的相位差也会改变，计数方向也会改变。

初始化完成后，编码器的脉冲计数是由定时器硬件完成的，不需要我们编写任何程序，我们只需要将定时器里的计数器的值读取出来即可，下面是实现过程：

```
int Read_Encoder(u8 TIMX)
{
    int Encoder_TIM;
    switch(TIMX)
```

```
{  
    case 2: Encoder_TIM= (short)TIM2 -> CNT; TIM2 -> CNT=0;break;  
    case 3: Encoder_TIM= (short)TIM3 -> CNT; TIM3 -> CNT=0;break;  
    case 4: Encoder_TIM= (short)TIM4 -> CNT; TIM4 -> CNT=0;break;  
    default: Encoder_TIM=0;  
}  
    return Encoder_TIM;  
}
```

2.5 蓝牙程序设计

蓝牙模块用于 APP 与小车之间的通信，能实现控制小车的运动转向与查看数据、PID 调参、使参数掉电保存等功能。蓝牙模块与手机之间用蓝牙通信，与单片机用串口 3 通信，相当于一个手机到单片机之间的桥梁。初始化由 main 函数的 uart3_init(9600)完成。

单片机接收蓝牙 APP 的数据串口 3 的中断服务函数里实现，位置在 usart3.c 文件中 void USART3_IRQHandler(void)函数，在这里不展开。单片机发送数据到蓝牙 APP 的实现在 main.c 中的 APP_Show()函数里，在这里不展开，在蓝牙 APP 的帮助说明里有详细的使用方法。

下面讲解接收到蓝牙 APP 数据后如何控制小车转向，其函数实现如下：

```
void Get_RC(void)  
{  
    static u8 TurnFlag=0;  
    static float turn;  
    //接收到 app 控制信号，读取 1 次机械中值  
    if((Flag_Right||Flag_Left)&&TurnFlag==0)  
    {  
        turn = balance_point;  
        TurnFlag = 1;  
    }  
    //左右控制  
    if(Flag_Left)        balance_point = turn + LeftControl;  
    else if (Flag_Right) balance_point = turn - RightControl;  
    //APP 无控制，机械中值复位  
    if(Flag_Right==0&&Flag_Left==0&&TurnFlag==1)  
    {  
        balance_point = turn;  
        TurnFlag = 0;  
    }  
}
```

控制小车的转向是通过修改小车的机械中值来实现的，当 app 接收到转向数据后，程序会先保存当前小车的机械中值，再判断是需要向哪一边转向，在原本的机械中值加上对应方向的偏差，即可完成转向；在无 app 控制后，把中值恢复成原始值，即可停止转向。

2.6 ADC 采集电池电压与电位器程序设计

小车的模拟量采集均使用 ADC1 来完成，其中 ADC 的初始化由 main 函数中 Adc_Init() 完成。配置的具体过程这里不展开，配置结果为 ADC 数据右对齐，非扫描模式，非连续转换模式，由软件触发采集。要完成多通道采集的同时而不使用扫描模式，需要对用户提供一个接口，当传入需要采集的通道时再开启对应的通道扫描，该函数实现过程如下：

```
u16 Get_Adc(u8 ch)
{
    //设置指定 ADC 的规则组通道，一个序列，采样时间
    ADC-RegularChannelConfig(ADC1, ch, 1, ADC_SampleTime_239Cycles5);
    ADC-SoftwareStartConvCmd(ADC1, ENABLE); //使能指定的 ADC1 的软件转换启动功能
    while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC)); //等待转换结束
    return ADC_GetConversionValue(ADC1); //返回最近一次 ADC1 规则组的转换结果
}
```

该函数的实现原理是：在 ADC 的非扫描模式下，传入用户需要采集的通道，函数内会将该通道设置为采集序列 1，然后软件触发开启转换，返回结果。因此，两个电位器的采集和电池电压的采集只需要传入对应的通道即可。

电池电压采集结果与实际电池电压关系：

由 1.7 小节可知，输入的电压通过串联 10k 和 1k 的电阻分压后接入单片机的采集引脚。由简单的电路分析可知，串联电路电流相等，内部的电压关系如下： $U_{\text{实际电压}} = 11 * U_{\text{ADC}}$ 。所以将采集回来的电压乘以 11 倍即可获得电池电压。由于 ADC 的数据寄存器是 12 位的，其最高能测量的电压为 3.3V，所以最终电池电压的计算公式为：

$$U_{\text{实际电压}} = \text{Get_Adc}(\text{Battery_Ch}) * 3.3 * 11 / 4096$$

其余两个电位器原理与电池电压采集一样，通过一个线性关系完成机械中值和舵机初始位置的调节，在程序中实现过程如下：

```
舵机调节：SERVO_BIAS=(Get_Adc(Adjust_Ch)-1024)/12;
机械中值调节：BalancePoint_Offset=(5.06f/1024.0f)*(Get_Adc(ZhongZhi_Ch)-1024);
```

3. 小车控制流程图

小车程序流程图可分为两部分，第一个是主程序的 `while(1)` 死循环部分，另一个是中断处理数据并控制部分。主程序程序流程图如图3-1所示。系统上电后，会对小车上所有的外设进行初始化，包括延迟函数配置、初始化对应的定时器实现对应的功能、ADC 初始化、模拟 SPI 初始化、模拟 IIC 初始化、MPU6050初始化和 DMP 库初始化等等。初始化完毕后进入 `while(1)` 死循环，在循环里会通过标志位来判断是发送 app 数据和刷新 OLED，还是发送数据到上位机显示波形（该标志位由按键控制）。

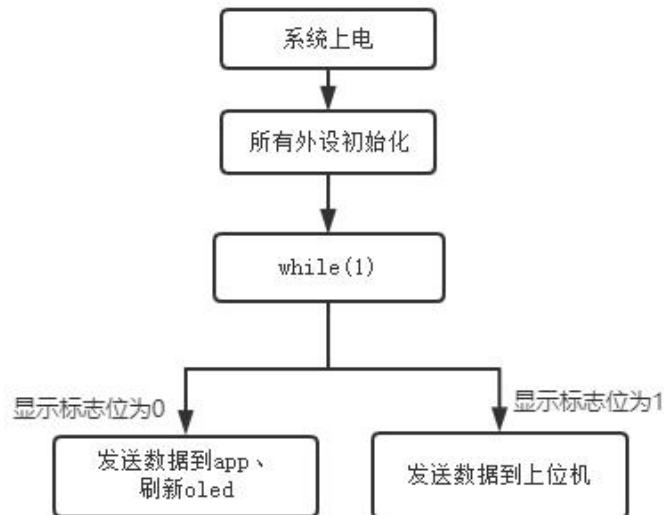


图3-1 主程序程序流程图

小车的控制部分在中断中完成，该中断由 MPU6050 的 INT 中断引脚提供。中断的频率可通过配置 MPU6050 的采样频率来改变，在本程序中中断频率是 200Hz (5ms 进1次中断)。其控制流程图如图3-2所示。控制部分包括所有传感器的数据读取、PID+LQR 的计算、电机与舵机的控制、接收蓝牙 app 的数据等内容。

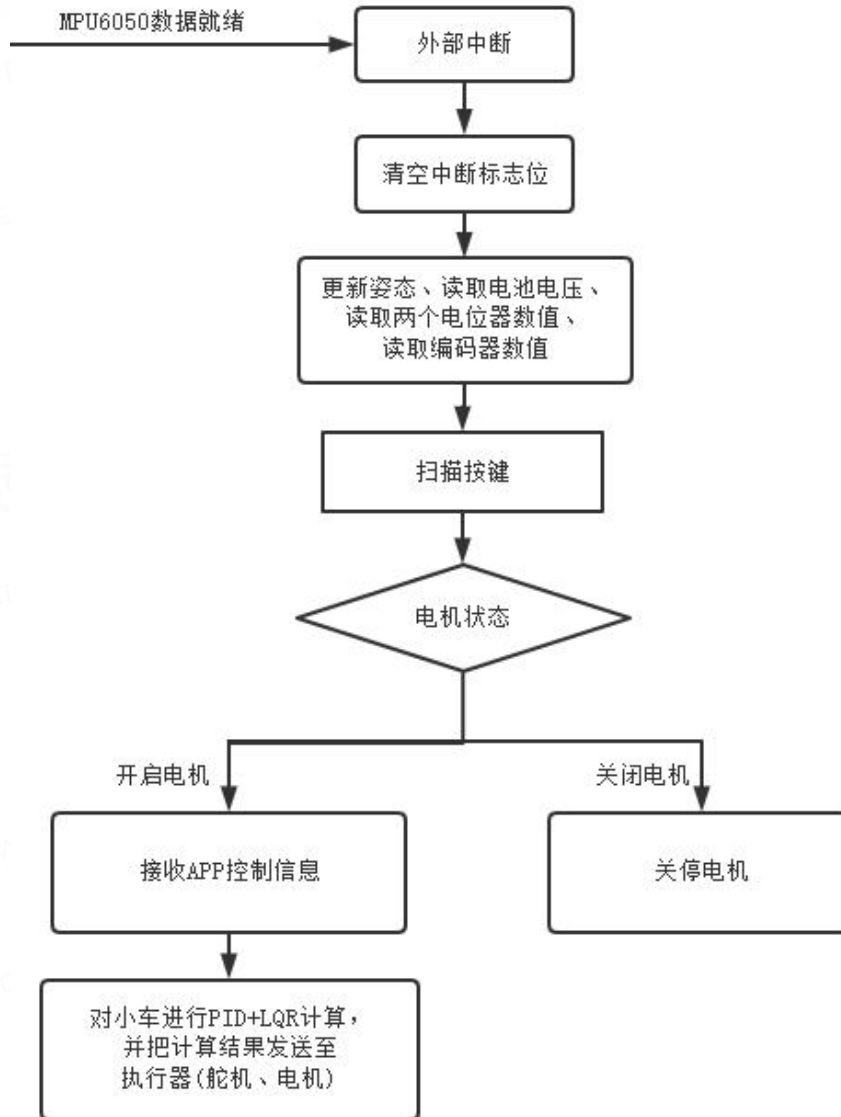


图3-2 中断任务程序流程图